Processor implementation

the big picture

processor has two main parts:

- datapath: carries out arithmetic and other operations, moves operands and results around
- control: selects, activates, sequences activities within the datapath

execution of a machine language instruction is broken up into steps; each step takes a single clock cycle to complete

... and so, the end of a clock cycle must cause one step to end, and a new step to begin, but how?

use storage elements (e.g. PC, other registers) that are implemented in such a way that the stored values can change only at the end of a clock cycle

Sequential circuits

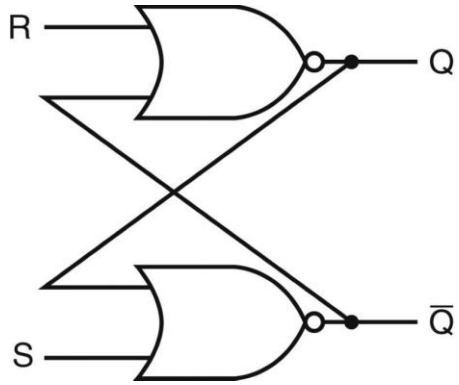
(reference: text Sections A.7, A.8, 4.2)

circuits we've seen so far have been combinational circuits, whose output is purely a function of the current input

the output of sequential circuits depends both on the current input, and on the current circuit "state" (value stored), as set by previous inputs

S-R latch

(from text Fig. A.8.1)



stored value is given by Q output

when S = 1, R = 0, stores a 1

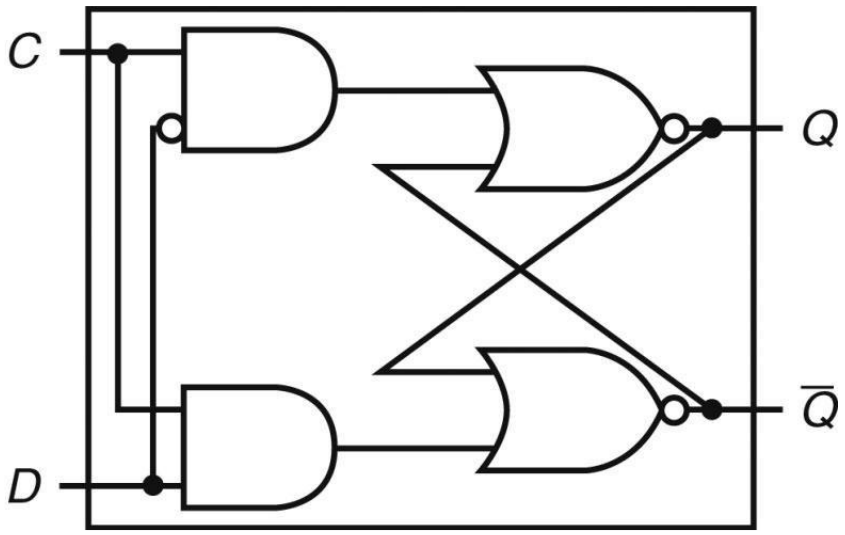
when S = 0, R = 1, stores a 0

when S = 0, R = 0, no change to the stored value

... how do we figure this out?

D latch

(from text Fig. A.8.2)

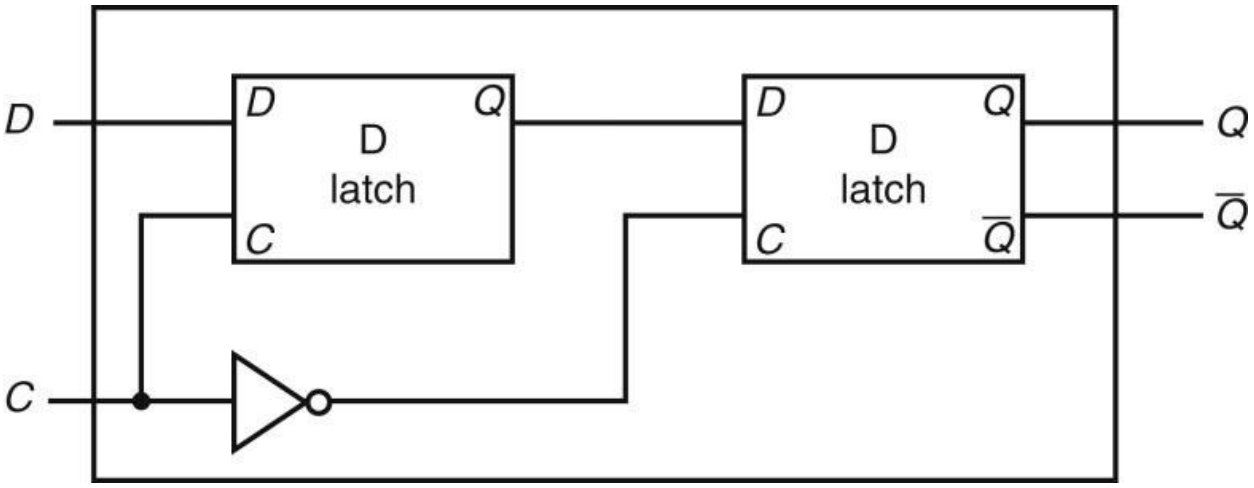


when C = 0, no change to the stored value

when C = 1, stores the D input value

D flip-flop

(from text Fig. A.8.4)



the D input value is stored only when the clock input C transitions from 1 to 0, marking the end of a clock cycle and the beginning of the next

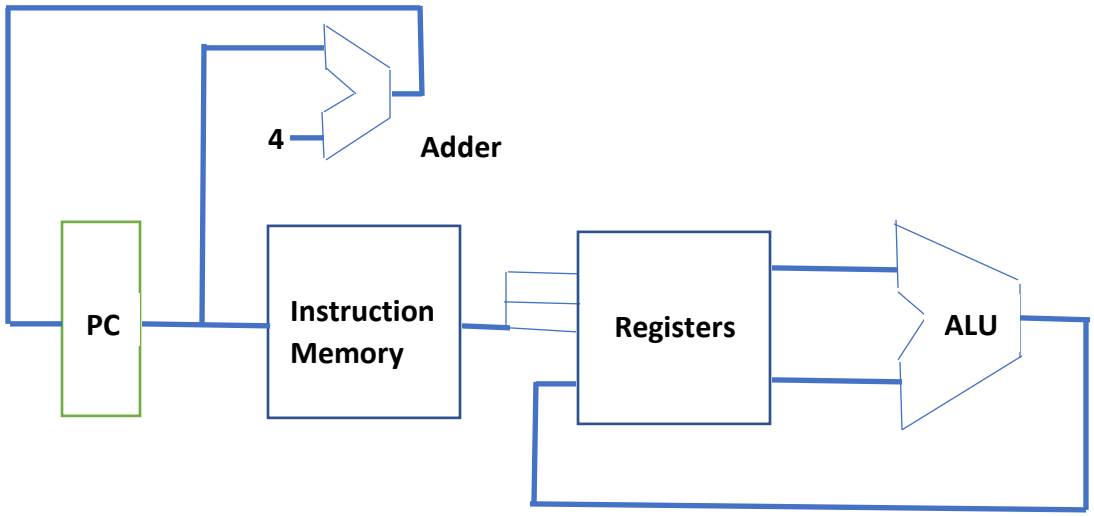
can build a 32-bit register whose contents can change only at the end of a clock cycle, using 32 of these D flip-flops

A simple, single clock cycle datapath

(reference: text Sections 4.1, 4.3, 4.4)

start with a datapath that can only execute the RISC-V "add", "sub", "and", and "or" instructions

(we have designed an ALU that can perform these operations)



includes: PC, adder, instruction memory, "register file", ALU

why is this a "single clock cycle" datapath?

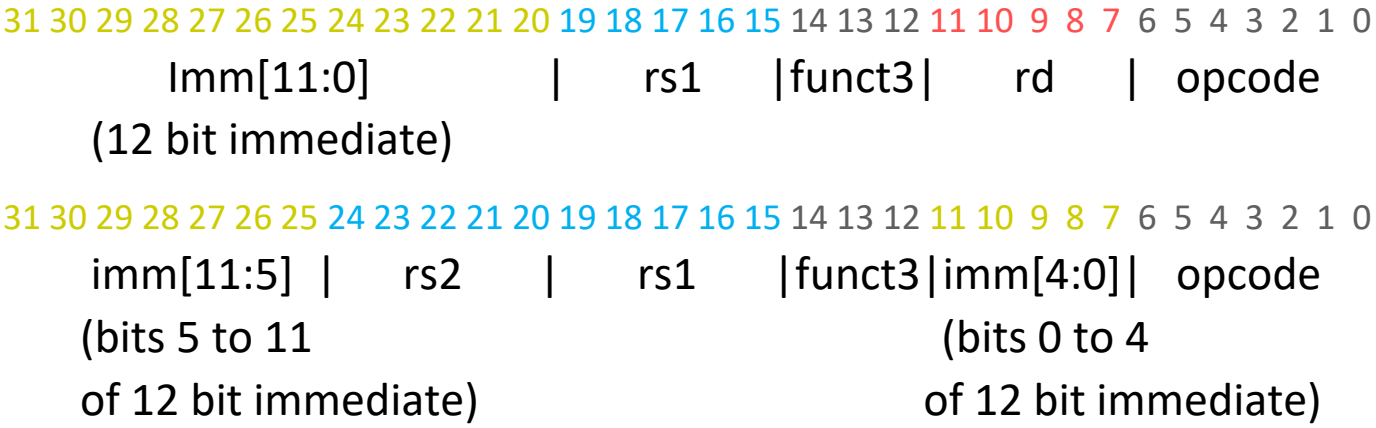
for these operation times: memory - 2 ns, ALU - 1 ns, adder - 1 ns, register file (read or write) - 0.5 ns, how long would the clock cycle time need to be?

why do we need a separate adder, instead of just using the ALU to add 4 to the PC?

datapath enhancements to support “lw”/“sw” and “beq”

initial datapath supported only R-type format instructions; “lw” uses the I-type format, “sw” uses the S-type format, and “beq” uses the B-type format

lw/sw



need a “data memory”

- add data memory component, with read/write control lines

need to form a 32-bit immediate operand

- add “immediate generation” unit

need to use as one of the inputs to the ALU the immediate operand rather than a second register

- add new multiplexor

for “lw”, need to write the word that was read from memory into a register, rather than the result of the ALU operation as with the R-type instructions

- add new multiplexor

for “sw”, need to disable writes into register file

- add “write control line”

(from text Fig. 4.10)